

Luiss

Libera Università Internazionale
degli Studi Sociali Guido Carli

Iteration

Introduction to Computer Programming

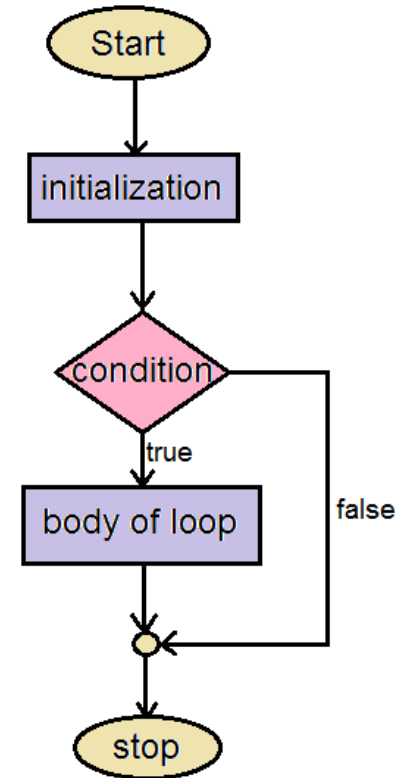
Management and Artificial Intelligence

LUISS



Iteration

- **Iteration** is the process where a set of instructions (or statements) is executed repeatedly for a specified number of time or until a condition is met.
- Like branching, these statements also **alter the control flow** of the program.



Anatomy of a looping mechanism.

while loops

```
while <condition>:  
    statement  
    statement  
    ...  
    statement
```

- <condition> evaluates to a Boolean (True or False)
 1. if <condition> is True, then execute all statements inside the while block
 2. evaluate <condition> again
 3. repeat until <condition> is False

Example

```
n = input("You are in the Lost Forest. Go Left or Right?: ")
while n == "Right":
    n = input("You are in the Lost Forest. Go Left or Right?: ")
print("You are out of the Lost Forest.")
```

for loops

```
for <variable> in range(<number>):  
    statement  
    statement  
    ...  
    statement
```

- each time through the loop, <variable> takes some value
- <variable> starts from the smallest value (usually, 1)
- next time, <variable> = <variable> + 1

The range() function

The range(start, stop, step) function returns a sequence of numbers, starting from start, and increments by step, and stops at stop-1. Keep in mind the following caveats:

- stop is mandatory
- step is optional: if not provided, it defaults to 1
- start is optional: if not provided, it defaults to 0

Examples:

```
mysum = 0
for i in range(7, 10):
    mysum = mysum + i
print(mysum)
```

24

```
mysum = 0
for i in range(5, 11, 2):
    mysum = mysum + i
print(mysum)
```

21

for vs. while loops

Iterate through sequence of numbers in `[0, 4)`:

- With a while loop:

```
n = 0
while n < 4:
    print(n)
    n = n + 1
```

0
1
2
3

- With a for loop:

```
for n in range(4):
    print(n)
```

0
1
2
3

Main differences

for loops

- **known** number of iterations
- can **end early** via break
- (natively) uses a **counter**
- you **can rewrite** a for loop using a while loop

while loops

- **unbounded** number of iterations
- can **end early** via break
- you can use a **counter**, but you must initialise it before the loop and increment it inside the loop
- you **may not be able to rewrite** a while loop using a for loop

More on for loops

for loops can be used to iterate over other data structures (e.g., lists or dictionaries*).

Example: print all functions in the math module that contain the word 'log';

```
import math
for f in dir(math):
    if 'log' in f:
        print(f)
print("Done!")
```

```
log
log10
log1p
log2
Done!
```

RECALL: dir() returns a list.

* We will see lists and dictionaries later in the course

Nested Loops

We know that branching (i.e., `if`) statements can be arbitrarily nested. The same holds for loops.

Example:

```
mysum = 0

for i in range(3):
    for x in range(2):
        mysum += i

print(mysum)
```

6

RECALL: `mysum += i` is the same as `mysum = mysum + i`

break statement

The break statement:

- immediately exits whatever loop is it in
- all further statements in the block are skipped
- exits only the innermost loop
- usually triggered at given condition

```
while <condition_1>:  
    while <condition_2>:  
        <statement_a>  
        break           # exits inner loop  
        <statement_b>   # never executed  
    <statement_c>
```

Example

```
mysum = 0

for i in range(5, 11, 2):
    mysum += i
    if mysum == 5:
        break
    mysum += i

print(mysum)
```

5

What's going on here?

Nested loops with `break`, an example:

```
mysum = 0

for i in range(30):
    for n in range(20):
        if n > 2:
            break
        mysum += i
    if i > 1:
        break

print(mysum)
```

9

What's going on here?

Warm-up exercises

- [Notebook with exercises on iteration \(no hints\)](#)
- [Notebook with exercises on iteration \(with pseudocode\)](#)

Exercise session

Find the exercises for the exam session: [here](#)

